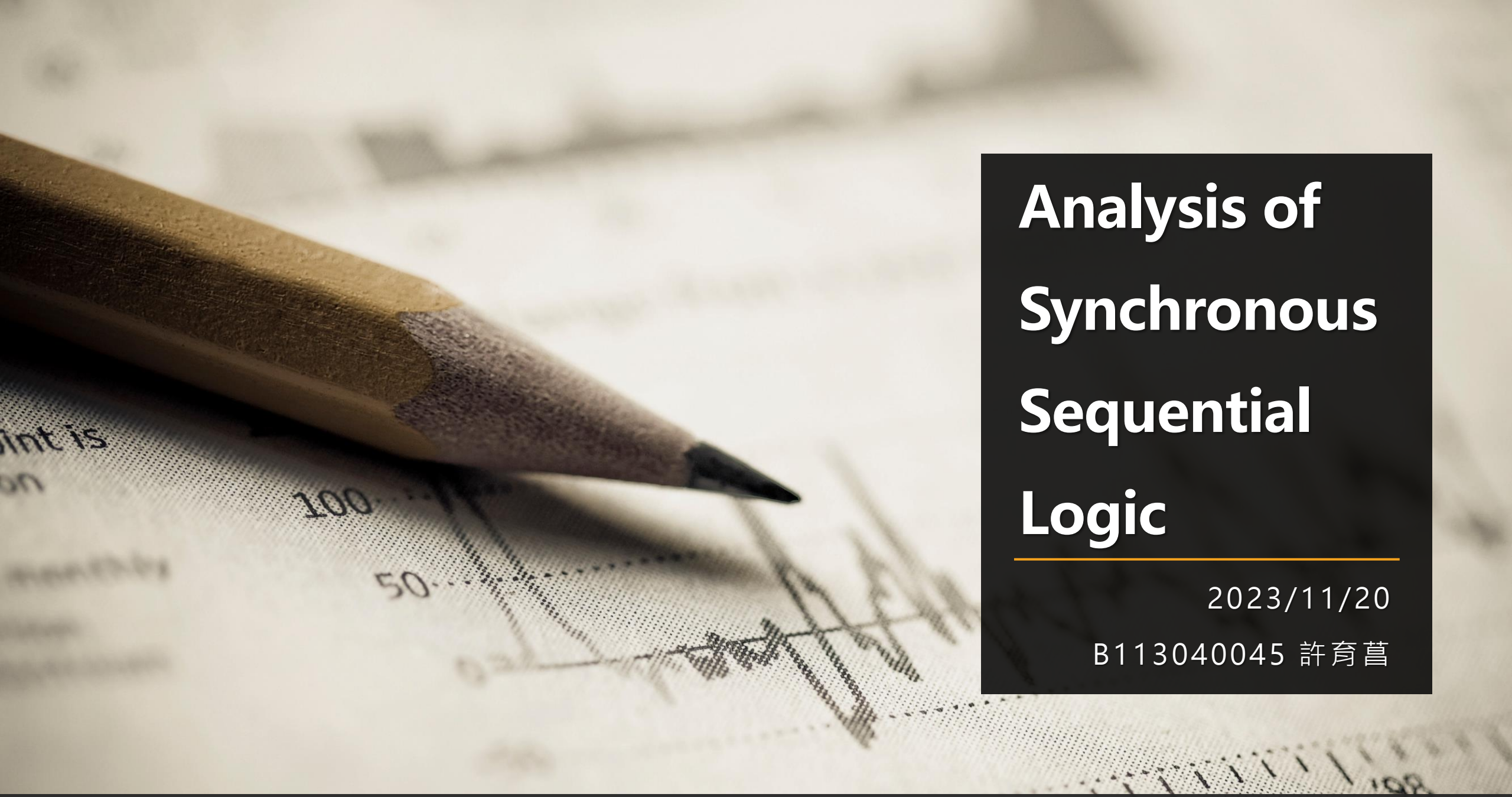
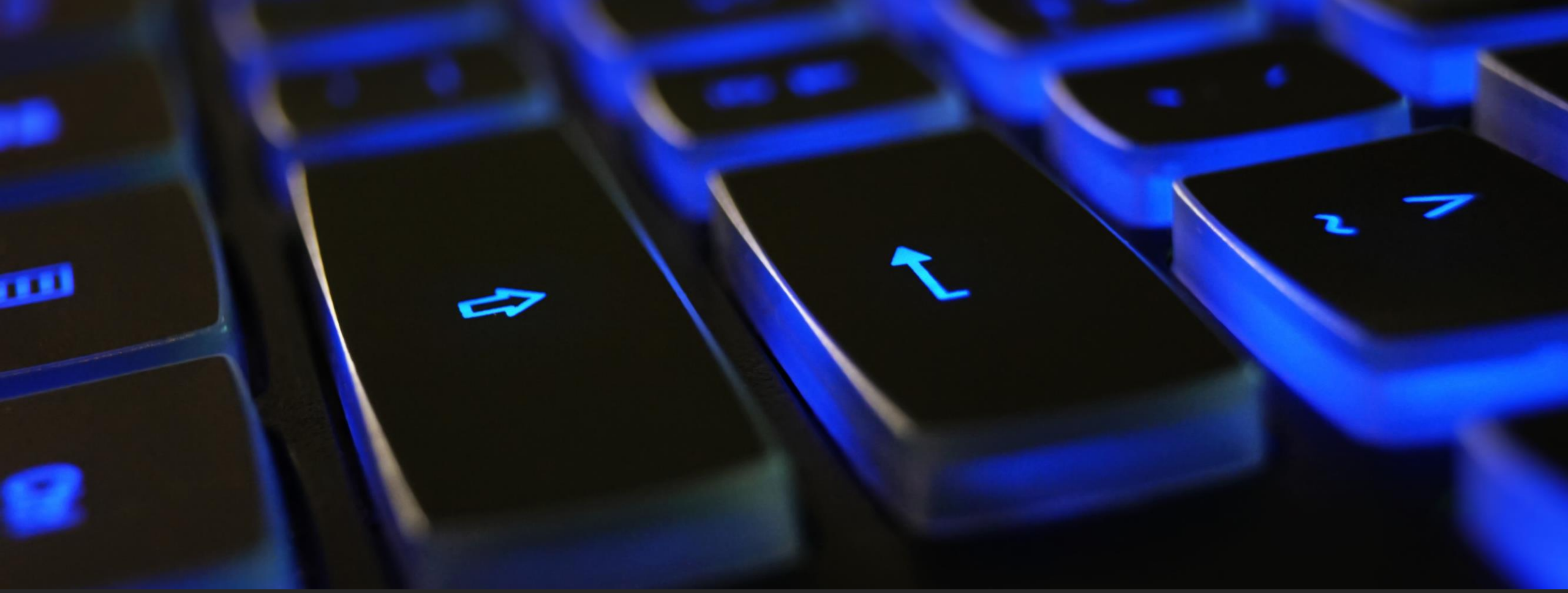


Analysis of Synchronous Sequential Logic

2023/11/20

B113040045 許育菖



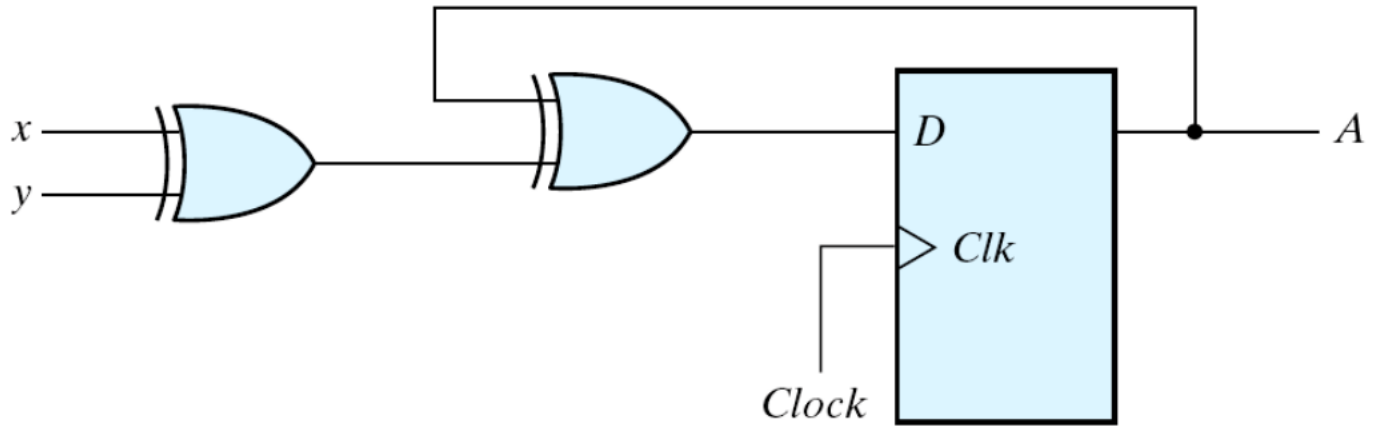
Exercise 1

Analysis of Sequential Circuit (D-FF)

題目說明

Simulate the sequential circuit shown in **Fig. 5.17**

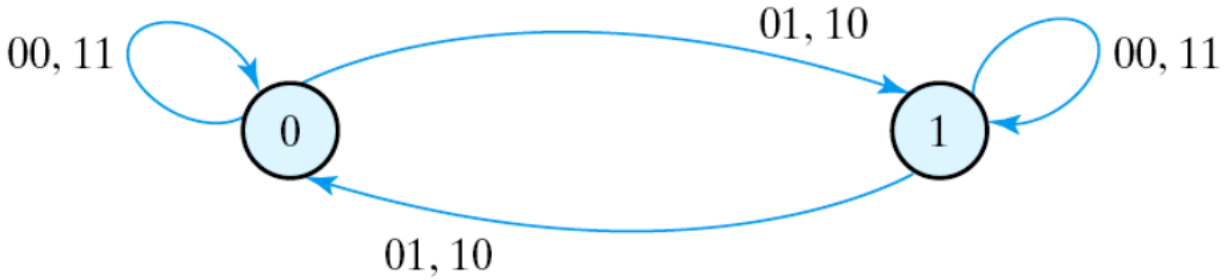
- Drive the input equation and state equation of the sequential circuit shown in Fig. 5.17(a)
- Drive the state table and the state diagram of the sequential circuit shown in Fig. 5.17(a)
- Write the Verilog HDL description of the state diagram (i.e., **Behavioral model**)
- Write the Verilog HDL description of the logic circuit diagram (i.e., **Structural model**)
- Write a Verilog HDL stimulus with a sequence of inputs: 00, 01, 11, 10. Verify that the response is the same for both descriptions **(first reset A to 0)**.



(a) Circuit diagram

Present state	Inputs		Next state
A	x	y	A
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(b) State table



(c) State diagram

Fig. 5.17 Sequential circuit with *D* flip-flop

Verilog code

logic circuit diagram

按照Fig5.17的電路圖進行設計，

其中D flip-flop獨立出來撰寫

```
// logic circuit diagram

`timescale 1ns / 1ps

module exercise1(A, x, y, clock, reset);
    output A;
    input x, y, clock, reset;

    wire xor1, D;
    xor(xor1, x, y);

    assign D = ( (!A) & xor1 ) | ( A & (!xor1) );

    flip_flop out(A, D, clock, reset);
endmodule
```

```
// D flip_flop()

`timescale 1ns / 1ps

module flip_flop(
    output reg Q,
    input D, clk, rst);

    always @ (posedge clk, negedge rst)
        if(rst == 0)
            Q <= 1'b0;
        else
            Q <= D;
endmodule
```

Verilog code

state diagram

依據Fig5.17(c)，根據當前處於哪一個state及x、y的值得出next state的值。

state變數以兩位的二元陣列表示。

```
// state diagram

timescale 1ns / 1ps

module exercise1_stage(A, x, y, clock, reset);
    output A;
    input x, y, clock, reset;

    reg [1:0] state;
    parameter S0 = 2'b00, S1 = 2'b01;
always @ (posedge clock, negedge reset)
    if(reset == 0)
        state <= S0;
    else
        case(state)
            S0: if( (x == 1 && y == 1) || (x == 0 && y == 0) )
                    state <= S0;
                else
                    state <= S1;
            S1: if( (x == 1 && y == 1) || (x == 0 && y == 0) )
                    state <= S1;
                else
                    state <= S0;
        endcase
  assign A = (state == S1);
endmodule
```

Testbench

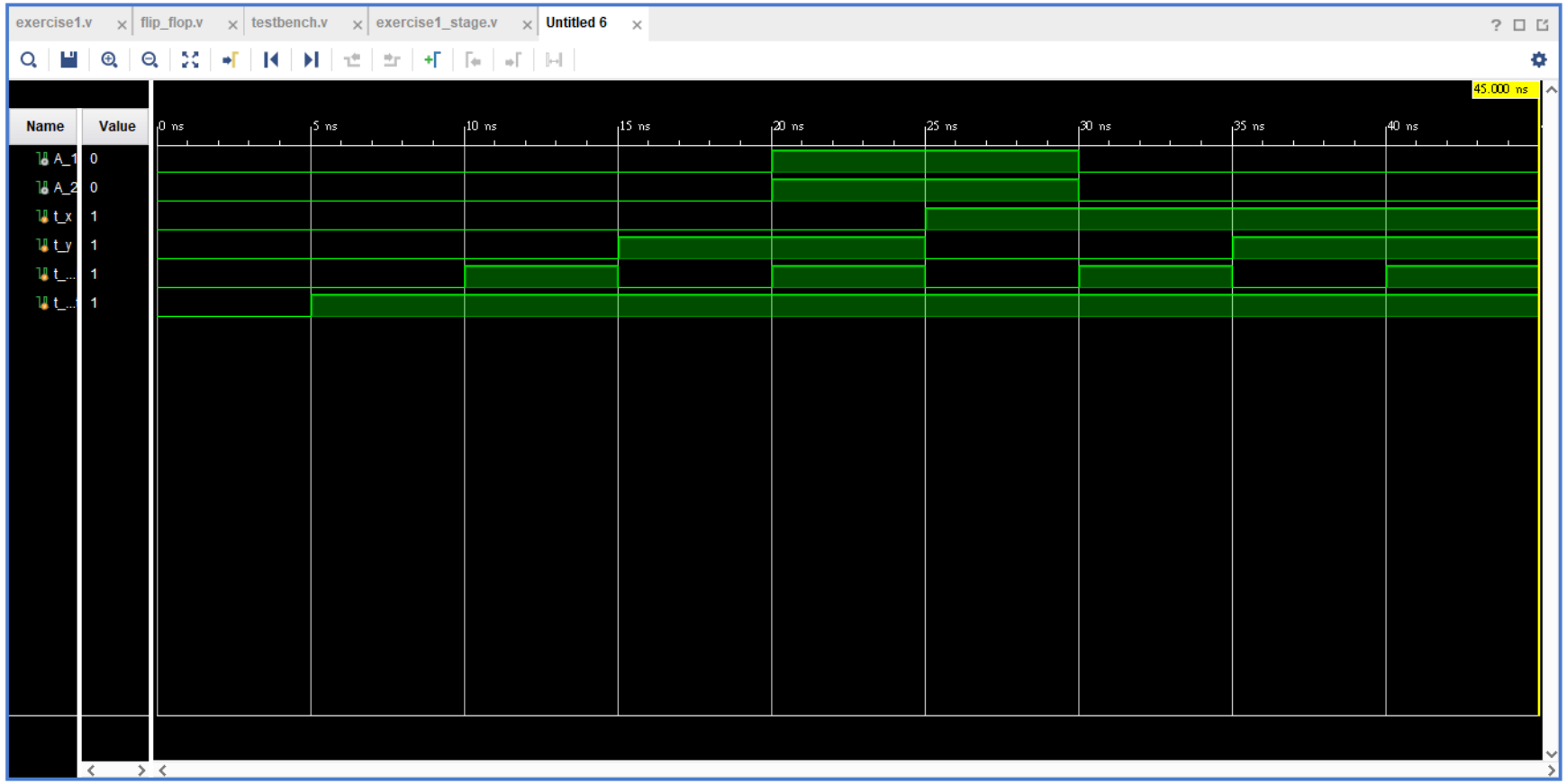
分成兩條時間軸：

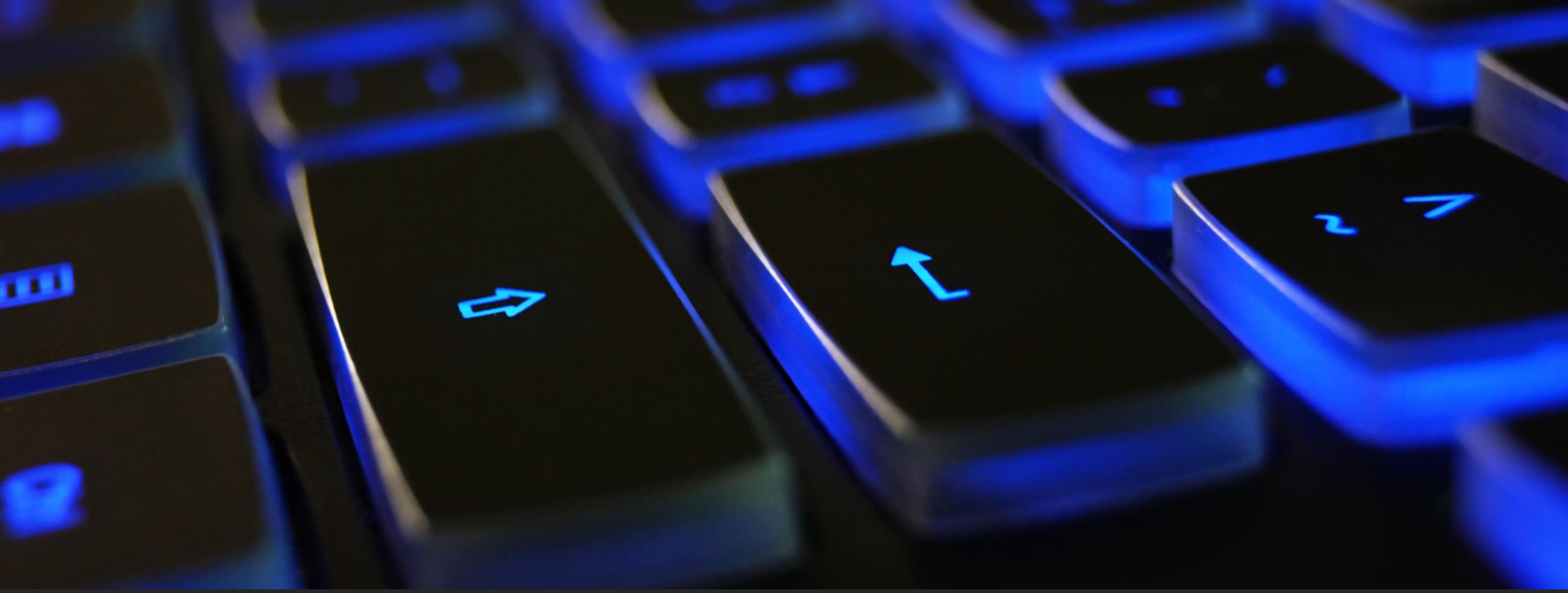
第一條處理reset和clock值的變化。

第二條處理x、y的輸入值。

```
`timescale 1ns / 1ps

module testbench();
    wire A_1, A_2;
    reg t_x, t_y, t_clock, t_reset;

    exercise1 UO (A_1, t_x, t_y, t_clock, t_reset);
    exercise1_stage U1 (A_2, t_x, t_y, t_clock, t_reset);

    initial #45 $finish;
    initial begin
        t_reset = 0;
        t_clock = 0;
    #5  t_reset = 1;
    repeat(8)
    #5  t_clock = ~t_clock;
    end
.
.
.
initial begin
        t_x = 0;  t_y = 0;
    #15 t_x = 0;  t_y = 1;
    #10 t_x = 1;  t_y = 0;
    #10 t_x = 1;  t_y = 1;
  end
endmodule
```

實驗結果


exercise1.v
flip_flop.v
testbench.v
exercise1_stage.v
Untitled 6
45.000 ns
Name
Value
A_1 0
A_2 0
t_x 1
t_y 1
t_... 1
t_... 1
0 ns
5 ns
10 ns
15 ns
20 ns
25 ns
30 ns
35 ns
40 ns



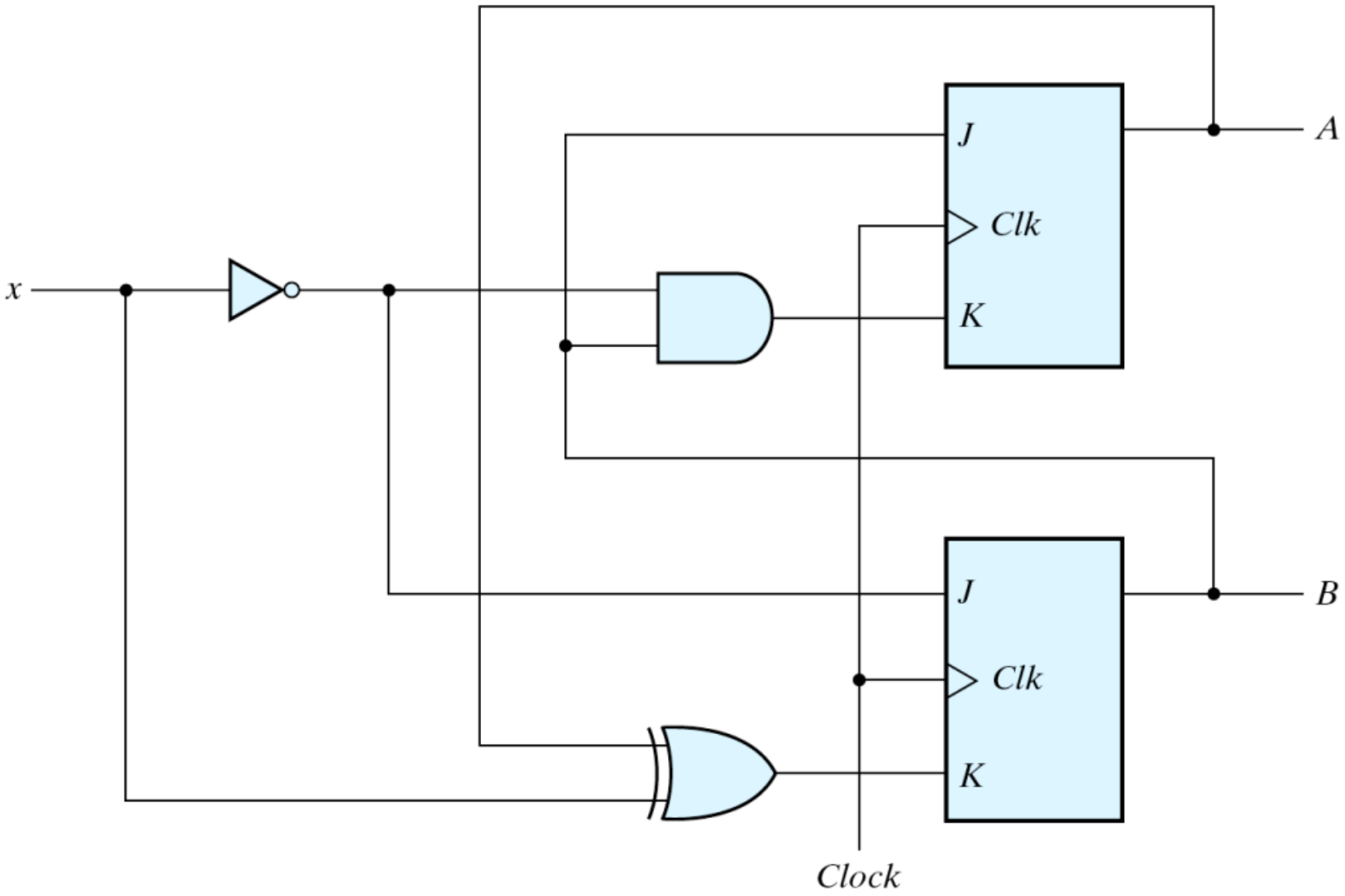
Exercise 2

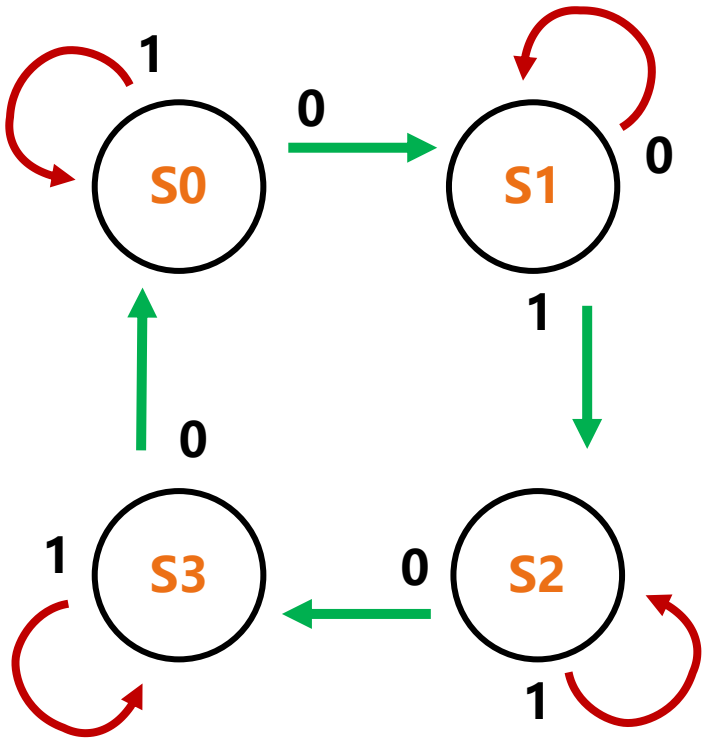
Analysis of Sequential Circuit (JK-FF)

題目說明

Simulate the sequential circuit shown in **Fig. 5.18**

- Drive the input equation and state equation of the sequential circuit shown in Fig. 5.18
- Drive the state table and the state diagram of the sequential circuit shown in Fig. 5.18
- Write the Verilog HDL description of the state diagram (i.e., **Behavioral model**)
- Write the Verilog HDL description of the logic circuit diagram (i.e., **Structural model**)
- Write a Verilog HDL stimulus with a sequence of inputs: 0, 1, 0, 0.
- Verify that the response is the same for both descriptions **(first reset A and B to 0)**.





state邏輯圖
(由logic circuit diagram的結果)

Fig. 5.18 Sequential circuit with *JK* flip-flop

Verilog code

logic circuit diagram

按照Fig5.18的電路圖進行設計，並將其中的JK flip-flop獨立出來撰寫

```
// logic circuit diagram

`timescale 1ns / 1ps

module exercise2(
    output A, B,
    input x, clock, reset
);
    wire and1, xor1;

    assign and1 = (!x) & B;
    assign xor1 = ( (!x) & A ) | ( x & (!A) );

    flipflop f1 (B, (!x), xor1, clock, reset);
    flipflop f2 (A, B, and1, clock, reset);

endmodule
```

```
// JK flip-flop()

`timescale 1ns / 1ps

module flipflop(
    output reg Q,
    input J, K, clk, rst);

    always @ (posedge clk, negedge rst)
        if(rst == 0)
            Q <= 1'b0;
        else
            Q <= ( J & (!Q) ) | ( (!K) & Q );
endmodule
```

Verilog code

state diagram

依據logic circuit diagram的結果，先畫出電路的state邏輯圖，再根據當前處於哪一個state及x的值得出next state的值。

state變數以兩位的二元陣列表示。

```
// state diagram

`timescale 1ns / 1ps

module exercise2_state(
    output A, B,
    input x, clock, reset
);

    reg [1:0] state;
    parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
always @ (posedge clock, negedge reset)
    if(reset == 0)
        state <= S0;
    else
        case(state)
            S0: if(x == 0) state <= S1;
                else state <= S0;

            S1: if(x == 1) state <= S2;
                else state <= S1;

            S2: if(x == 0) state <= S3;
                else state <= S2;

            S3: if(x == 0) state <= S0;
                else state <= S3;
        endcase
    assign A = (state == S3 || state == S2);
    assign B = (state == S1 || state == S3);
endmodule
```

Testbench

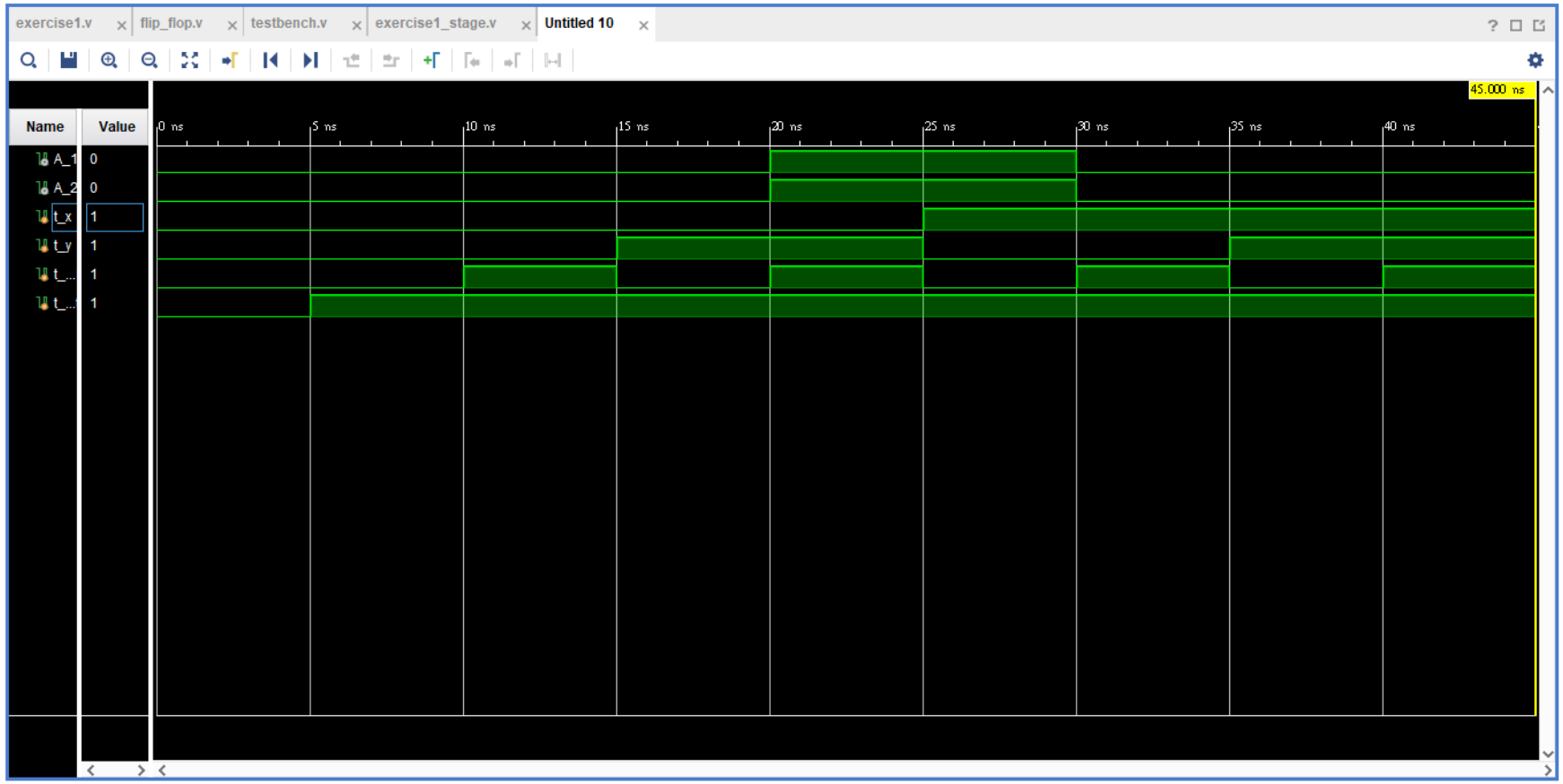
分成兩條時間軸：

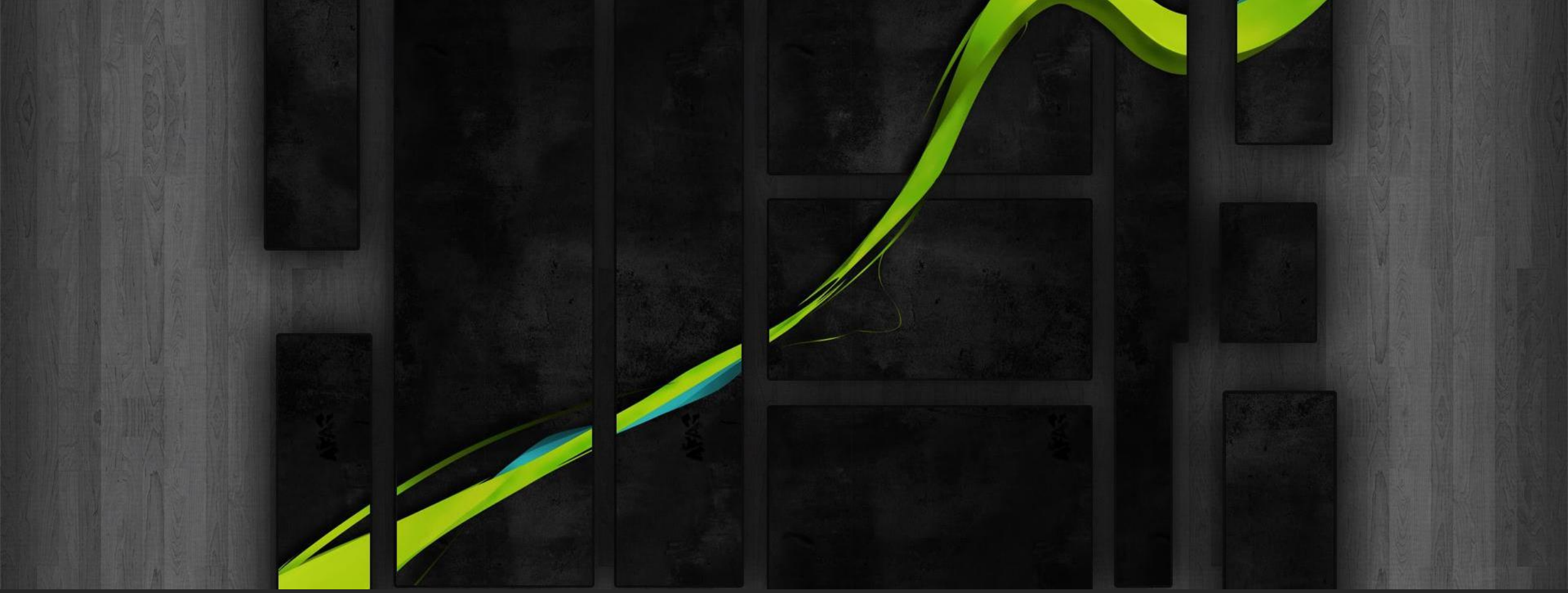
第一條處理reset和clock值的變化。

第二條處理x、y的輸入值。

```
`timescale 1ns / 1ps

module testbench();
    wire A_1, A_2, B_1, B_2;
    reg t_x, t_clock, t_reset;

    exercise2 UO (A_1, B_1, t_x, t_clock, t_reset);
    exercise2_state U1 (A_2, B_2, t_x, t_clock, t_reset);

    initial #45 $finish;
    initial begin
        t_reset = 0;
        t_clock = 0;
    #5  t_reset = 1;
    repeat(8)
    #5  t_clock = ~t_clock;
    end
.
.
.
.
.
.
initial begin
        t_x = 0;
    #15 t_x = 1;
    #10 t_x = 0;
    #10 t_x = 0;
  end
endmodule
```

實驗結果


exercise1.v
flip_flop.v
testbench.v
exercise1_stage.v
Untitled 10
45.000 ns
Name
Value
A_1
0
A_2
0
t_x
1
t_y
1
t_...
1
t_...
1
0 ns
5 ns
10 ns
15 ns
20 ns
25 ns
30 ns
35 ns
40 ns



實驗心得

Experiment reflection

此次實驗實作了兩種正反器，分別是D flip-flop以及JK flip-flop。

總體而言，我覺得這次實驗比上次還要不好實作。尤其是在電路中正反器的部分，但並不是像以往的Verilog code寫不出來，而是我搞不太清楚Testbench中reset和clock這兩個變數的用途，以及為甚麼可以這樣用。這讓我在撰寫Testbench的時候很沒把握自己的寫法是不是對的。

然後在state diagram的部分，我這次實驗室直接從logic circuit diagram中得到的結果去推出state邏輯圖的，並不是直接從電路圖推出來，之後要再找時間練習。